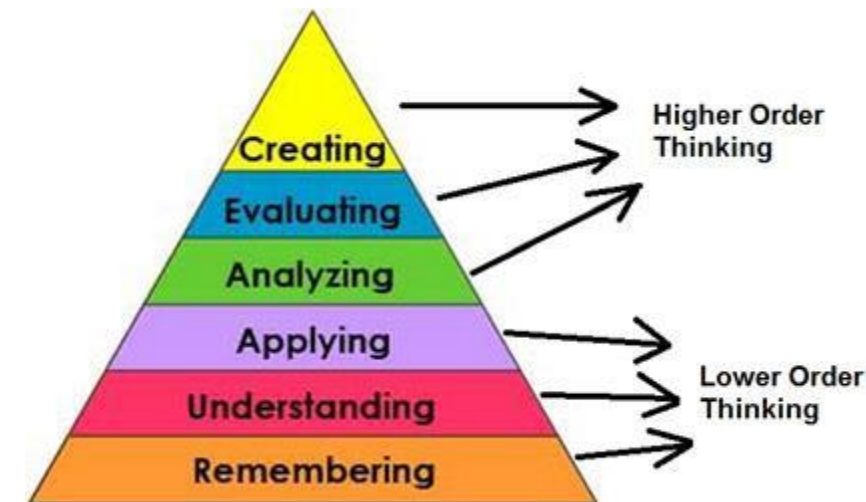


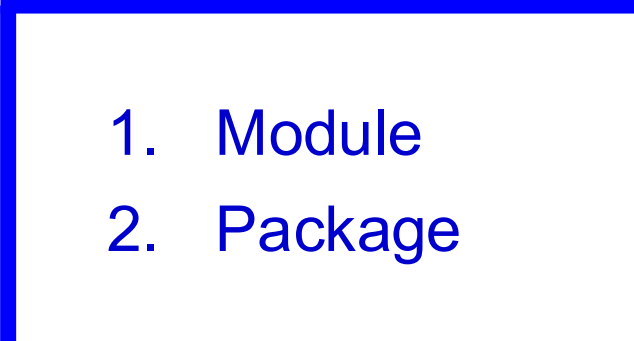
Kỹ Thuật Lập Trình Python

CHƯƠNG 6 MODULE - PACKAGE



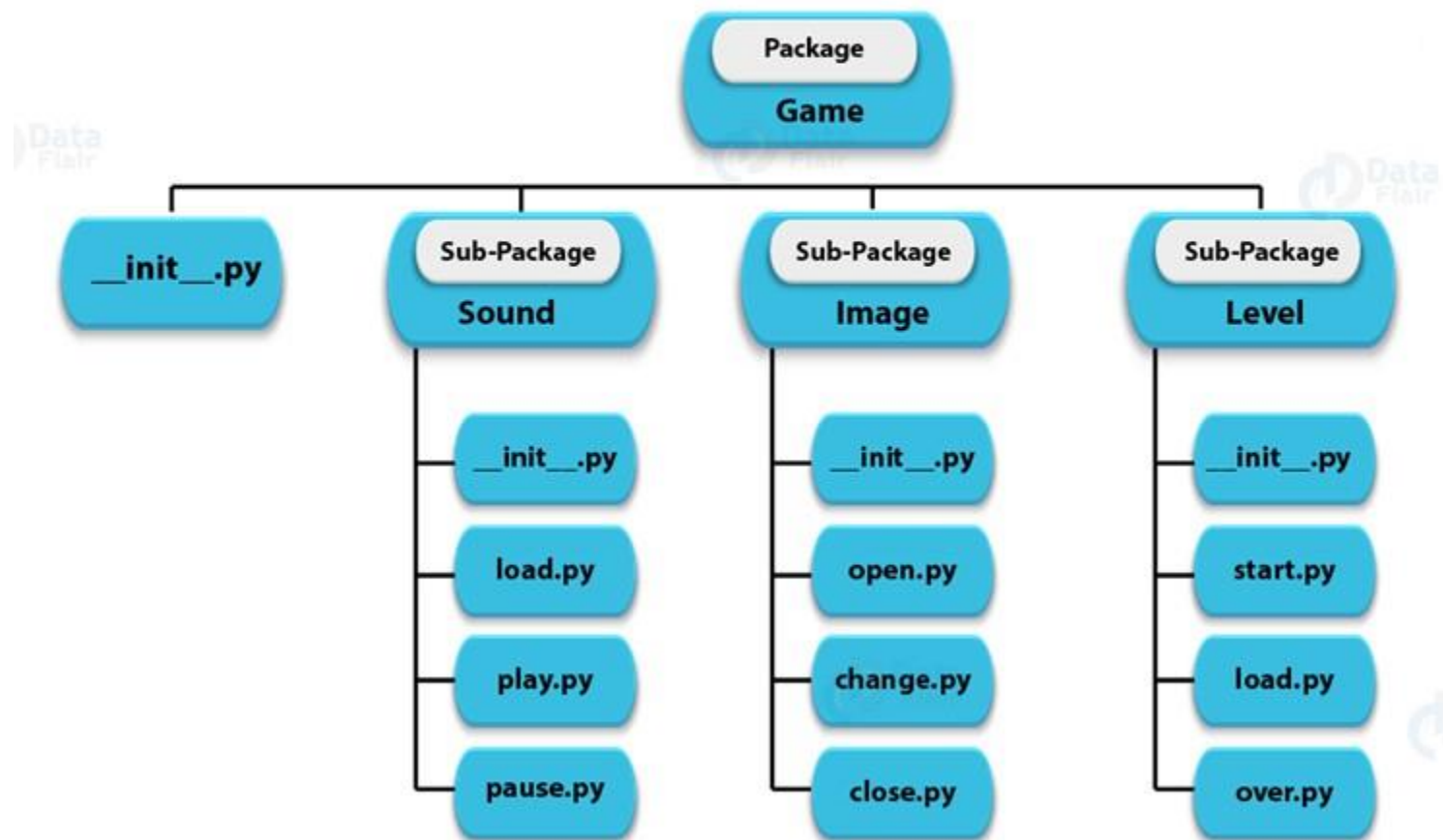
- Vận dụng module quản lý các hàm, biến số...
- Áp dụng tổ chức project bằng package.



- 
1. Module
 2. Package

Giới thiệu cấu trúc chung của Module và Package

Package Module Structure



Giới thiệu module

- Là một file lưu trữ mã nguồn (**source code**) của Python, được lưu với dạng **<tên_file.py>**.
- module (file) chứa biến, câu lệnh, hàm, class...
- Ý tưởng dùng mô-đun là để **chia nhỏ** các chương trình lớn thành các file nhỏ để dễ dàng quản lý,
- Dễ dàng tối ưu các đoạn mã code nếu cần.

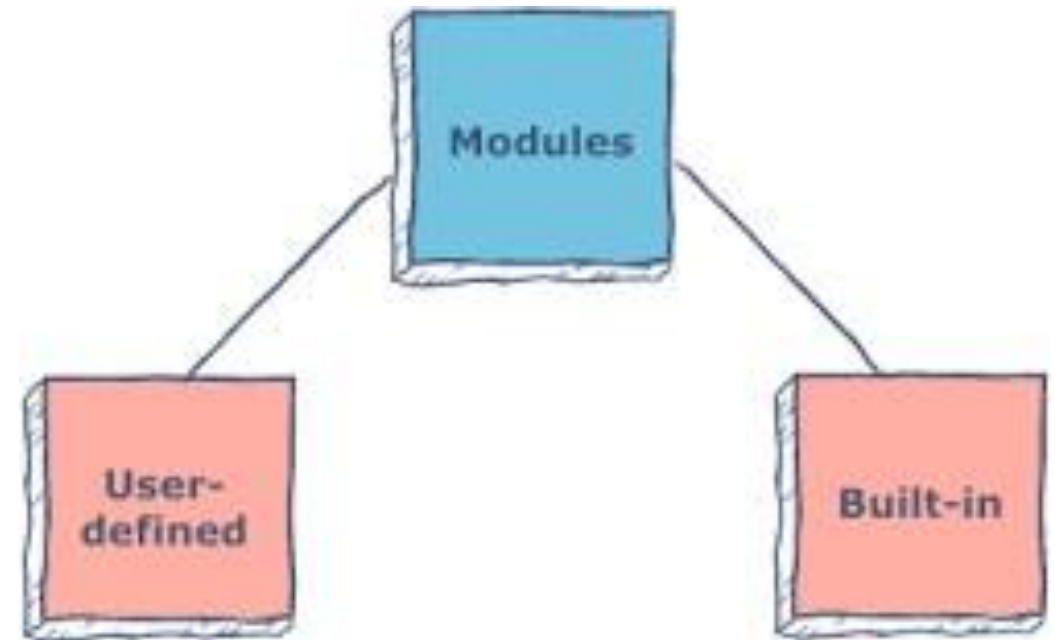
Giới thiệu module

- Sử dụng module khi chương trình có phạm vi thực hiện lớn, tránh việc lưu trữ mã nguồn chỉ duy nhất một file.
- Chia nhỏ thành từng tính năng, nhiệm vụ, và phải được lưu trữ chúng thành các module.
- module có source file có đuôi là **.py**

module

Có hai loại module

- Standard module (**Built-in**)
- Module tự định nghĩa (**User-defined**)



Đọc thêm ref.: <https://data-flair.training/blogs/python-modules/>

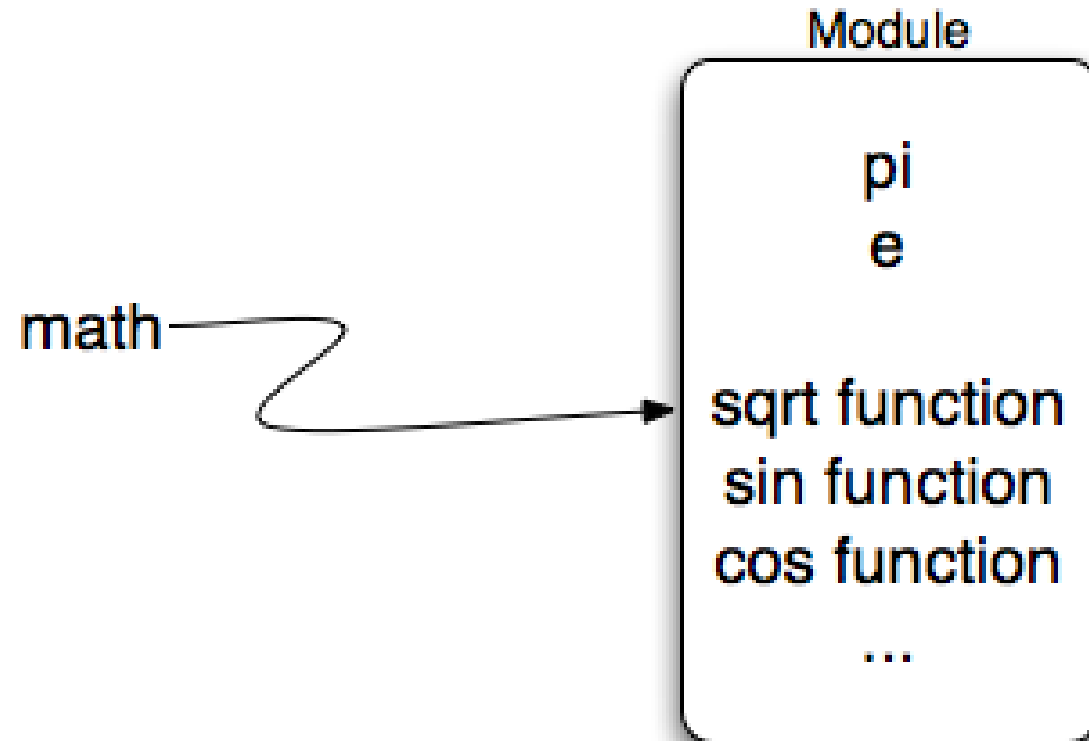
Có hai loại module

- **Standard module**: là các module chuẩn, có sẵn/mặc định trong Python.
- Ví dụ như: math, array, numbers, random... Có thể xem chi tiết toàn bộ các module có sẵn trong Python tại đường dẫn sau: <https://docs.python.org/3/py-modindex.html>
- **Module tự định nghĩa**: là các module do lập trình tự tạo thành để lưu trữ các mã nguồn theo mục đích tính năng công việc của lập trình viên.

Danh sách các built-in module

- Ref: <https://docs.python.org/3/py-modindex.html>

Ví dụ



Sử dụng built-in module bằng cách nào?

- Sử dụng từ khóa `import` để trỏ đến các module

Minh họa việc import một mô-đun có sẵn:

```
import math as m
import numpy as np
import pandas as pd
import statistics as st
import matplotlib.pyplot as plt
import seaborn as sns
import arcgis as gis
from IPython.display import display
```

Cú pháp import mô-đun có sẵn

- `import name_module1, name_module2..., module_n`

Cách gọi hàm từ một module

- `<name_module>.<function>`

Ví dụ

```
import math, random  
print("Tính Cos(30) =", math.cos(30))  
print("Random một giá trị ngẫu nhiên", random.random())
```

Kết quả:

```
Tính Cos(30) = 0.15425144988758405  
Random một giá trị ngẫu nhiên 0.9148759360621969
```

module có sẵn

- Từ khóa **as** qui định tên định danh mới cho mô-đun.
- Ví dụ:

```
import math as m, random as r
print("Tính Cos(30) =", m.cos(30))
print("Random một giá trị ngẫu nhiên", r.random())
```

Cú pháp from...import

- Cú pháp “**from ... import**” để sử dụng một hàm nào đó cụ thể trong mô-đun

from module **import** function1, function2, function3,...

- Hoặc, **from** module **import** * (sử dụng tất cả những gì có trong module và không import các hàm bắt đầu bằng ký tự “_”).

GV minh họa

- Minh họa trong PyCharm

package?

- Là việc tạo các thư mục để lưu trữ các mô-đun vào trong thư mục đó.
- Nếu chương trình của chúng ta có nhiều mô-đun liên quan nhau thì nên được quản lý cùng một Package.
- Trong thư mục này sẽ có một file đặc biệt có tên là `<__init__.py>`

Cách tạo package

- Tạo một thư mục dùng để làm package.
- Trong thư mục mới vừa tạo, chúng ta tạo một file Python có tên là `<__init__.py>` (file trống, không cần ghi thêm bất kỳ đoạn mã gì).
- Bất kỳ thư mục nào có chứa file `<__init__.py>` thì đó chính là một package trong Python.
- Lưu trữ toàn bộ các mô-đun mà chúng cần lưu trữ vào thư mục package đó.

Cách import package

- Để sử dụng các mô-đun trong một package, chúng ta sử dụng cú pháp sau:
- `from <package_name> import <module_names>`

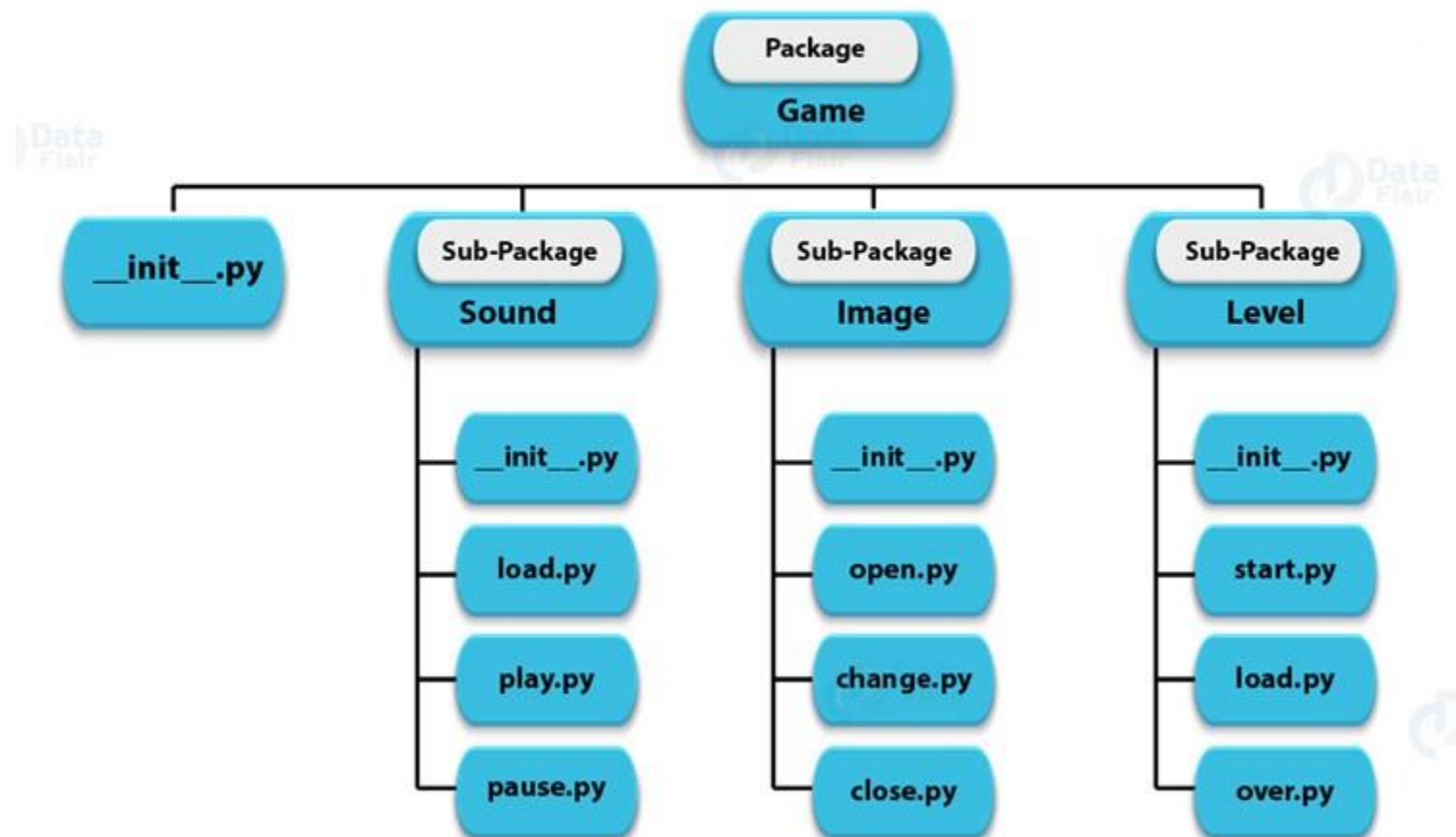
Ví dụ

```
from illustrating_package import illustrating_module as im  
  
im.printing_pos_UPPER(["trường", "đại", "học", "công",  
"nghệ", "thông", "tin"])
```

Kết quả:

```
[(0, 'TRƯỜNG'), (1, 'ĐẠI'), (2, 'HỌC'), (3, 'CÔNG'), (4,  
'NGHỆ'), (5, 'THÔNG'), (6, 'TIN')]
```

Package Module Structure



Phân biệt thư viện – module – package

Thư viện - library

- Thư viện trong Python chứa các mô-đun và package, được thiết kế cung cấp các chức năng, công cụ để giải quyết vấn đề cụ thể.

Ví dụ

- Thư viện NumPy là một thư viện hỗ trợ tính toán khoa học và số học.

Cách kiểm tra

- Câu lệnh `pip list` để kiểm tra một thư viện có tồn tại

Phân biệt thư viện – module – package

Mô-đun - module

- Mô-đun là tập tin chứa mã Python dùng để định nghĩa biến, hàm và lớp.

Ví dụ

- Mô-đun math chứa các hàm, hằng số toán học như sin, cos, và pi...

Cách kiểm tra

- Câu lệnh `help()` hoặc `dir()` để kiểm tra các mô-đun.

```
print(help('math'))  
print(dir('random'))
```

Phân biệt thư viện – module – package

Gói - package

- Thư mục chứa các mô-đun liên quan.
- Tổ chức và nhóm các mô-đun có chức năng tương tự lại với nhau.

Ví dụ

- Gói numpy là một gói phổ biến trong thư viện NumPy.

Cách kiểm tra

- `dir()` liệt kê tất cả các thành phần (bao gồm mô-đun, gói con và biến) có sẵn trong package đó.
- `pip show package_name`



Thank you

Hỏi - Đáp





Bài 1 – Module tự định nghĩa

1. Tạo một file `math_utils.py` với các hàm:

```
def add(a, b): ...  
def subtract(a, b): ...  
def multiply(a, b): ...  
def divide(a, b): ...
```

2. Tạo file `main.py` và:

Import toàn bộ module (`import math_utils`)
Gọi từng hàm với dữ liệu mẫu.

3. Chạy và kiểm tra kết quả



Bài 2 – Sử dụng built-in module

1. Viết chương trình sử dụng math và random:

Tính căn bậc hai của 25

Tính cos của 60 độ (chuyển sang radian)

Sinh một số ngẫu nhiên từ 1 đến 100

2. Dùng import as để đặt bí danh cho module.

Bài 3 – Package đơn giản

1. Tạo cấu trúc:

`my_package/`

`__init__.py`

`geometry.py` # Tính diện tích, chu vi hình vuông, hình tròn

`converter.py` # Đổi đơn vị: $cm \rightarrow m$, $m \rightarrow km$

`main.py`

2. Trong `main.py`:

Import `geometry` và `converter` từ `my_package`

Gọi các hàm để test.

3. Chạy và kiểm tra kết quả.

Bài 4 – Package nhiều cấp

1. Tạo cấu trúc package giống slide “Game”:

```
Game/  
  __init__.py  
  Sound/  
    __init__.py  
    play.py  
  Image/  
    __init__.py  
    open.py
```

Mỗi file .py in ra thông báo khi được gọi.

2. Trong main.py, import và gọi hàm từ Game.Sound.play và Game.Image.open.



Bài 5 – Kết hợp thư viện bên ngoài

1. Cài thư viện numpy bằng pip.

2. Tạo file `matrix_ops.py`:

Tạo ma trận 3x3 ngẫu nhiên.

Tính định thức, ma trận nghịch đảo.

3. Tạo file `main.py` import từ `matrix_ops` và in kết quả.